

Implementing A Simple RMAP Initiator Using The STAR-System C++ API

STAR-Dundee's software stack, STAR-System, provides a consistent programming interface for accessing STAR-Dundee interface and router devices, including the STAR-Dundee SpaceWire PCI Mk2, PXI and PCIe devices and the STAR-Dundee SpaceWire Brick Mk3 and Router Mk2S devices.

The C++ API provided with STAR-System provides classes with methods for opening channels to devices and transmitting and receiving packets on those channels. It also provides classes to encapsulate and build Remote Memory Access Protocol (RMAP) packets and to obtain the values of the fields in an RMAP packet.

This application note explains how to create and use RMAP packets. The examples provided are then combined with the transmit and receive code demonstrated in the application note "**Transmitting And Receiving Packets Using the STAR-System C++ API**" to demonstrate how a very basic RMAP Initiator can be implemented, which transmits an RMAP read command packet and receives an RMAP read reply packet. Further information on the APIs and the functions provided can be found in the STAR-System documentation.

Creating RMAP Packets

The C++ API provides the following classes to encapsulate the different types of RMAP packets which can be built, transmitted and received within STAR-System:

```
using namespace stardundee::com::starsystem::rmap;
```

Class	Description
RMAPReadCommandPacket	An RMAP read command.
RMAPWriteCommandPacket	An RMAP write command.
RMAPReadModifyWriteCommandPacket	An RMAP read-modify-write command.
RMAPReadReplyPacket	An RMAP read reply, contains the data requested.
RMAPWriteReplyPacket	An RMAP write reply, acknowledges a write.
RMAPReadModifyWriteReplyPacket	An RMAP read-modify-write reply, contains the data requested and acknowledges the write.
RMAPPacketLibrary	Provides version information for the RMAP packet library and can be used to create RMAP packets from data received via a ReceiveOperation .

Each of the RMAP packet classes provides a [Create\(\)](#) method which can be used to populate the packet with the appropriate values, e.g. [CreateReadCommandPacket\(\)](#) is used to build a read command packet. These functions calculate the amount of memory required for the packet, allocate memory for the packet, then fill that memory with the packet. The properties of the packet to be built are specified as parameters. These properties can be retrieved by calling relevant methods on the packet object. The example below shows an RMAP read command packet being created:

```

RMAPReadCommandPacket readCommandPacket;
int result;
U8 targetAddress[] = {0, 254};
U8 replyAddress[] = {254};
const unsigned long targetAddressLength = sizeof(targetAddress);
const unsigned long replyAddressLength = sizeof(replyAddress);
const char alignment = 1, incrementAddress = 0;
const U8 key = 0x20, extendedReadAddress = 0;
const U16 transactionIdentifier = 0;
const U32 readAddress = 0x106, readLength = 4;

// build an RMAP read command packet to read 4 bytes
if (!readCommandPacket.CreateReadCommandPacket(targetAddress,
        targetAddressLength, replyAddress, replyAddressLength,
        incrementAddress, key, transactionIdentifier,
        readAddress, extendedReadAddress, readLength, alignment))
{
    cout << "Couldn't build the read command packet" << endl;
    return;
}

// \todo do something with the packet

// dispose of the packet
// \note The packet will be automatically disposed if required when the
//       RMAP packet object goes out of scope or is deleted, i.e. in the
//       object destructor, if it has not been manually disposed of.
readCommandPacket.Dispose();

```

Receiving RMAP Packets

RMAP commands can be transmitted and received in the same way as SpaceWire packets. A [TransmitOperation](#) can be used to transmit the data which contains the RMAP command and a [ReceiveOperation](#) can be used to receive the data which contains the RMAP reply or RMAP commands. The [GetRMAPPacketFromStreamItem\(\)](#) function on the [RMAPPacketLibrary](#) object can be used to extract RMAP objects from stream items received via a [ReceiveOperation](#). The first parameter to this function is an instance of the [Packet](#) stream item which is expected to contain an RMAP packet. The second parameter indicates whether to check if the packet is longer than expected. This can be used to check for unexpected bytes which follow the end of the RMAP packet. When the transmitter or receiver of the packet is only capable of sending packets that are multiples of 4 bytes in length, for example, there may be additional padding bytes at the end of the packet and so it may be necessary to ignore these bytes. The final parameter is an [RMAPPacket::RMAPStatus](#) value which is used to store the status of the attempt to extract the RMAP packet from the given stream item. The [GetRMAPPacketFromStreamItem\(\)](#) function will check whether the data contained in the given stream item contains a valid RMAP packet and set the status accordingly. The following checks are made:

- Whether the instruction field is valid:
 - The packet type is valid
 - The reserved bit of the packet type is 0
- Whether the packet length is correct (unless this is not being checked)
- Whether the protocol ID is the RMAP protocol ID
- Whether the reply address (if expected) is present
- Whether the data (if expected) is present
- Whether, in a read or read-modify-write reply, the reserved byte is 0
- Whether the target and reply addresses are terminated with a logical address

- Whether the header and (if present) data CRCs are correct
- Whether, in a read-modify-write command, the data and mask length is 0, 2, 4, 6 or 8

The following code extract shows a `Packet` stream item being retrieved from a completed `ReceiveOperation` and an attempt being made to extract an `RMAPPacket` object from this `Packet`.

```
Packet *pPacket;
RMAPPacket *pRMAPPacket;
RMAPPacket::RMAPStatus status;

// get packet from completed receive operation
pPacket = (Packet *)receiveOp.GetTransferItem(0);

// no packet received
if (!pPacket)
{
    cout << "Unable to get the stream item received." << endl;
    return;
}

// extract RMAP packet from the received stream item
pRMAPPacket = RMAPPacketLibrary::GetRMAPPacketFromStreamItem(pPacket,
                                                             1, &status);

// free the received stream item, now it has been processed
delete pPacket;

// if failed to extract valid RMAP packet from received stream item
if (!pRMAPPacket)
{
    cout << "A valid RMAP packet was not received. Status: " << status << endl;
    return;
}

// \todo do something with the RMAP packet
```

Getting the Value of RMAP Packet Fields

RMAP packet objects provide a number of functions to query the fields in an RMAP packet. The example below shows how to get the type, target address and key value of the RMAP packet received in the previous code extract:

```
U8 *pAddress, key;
unsigned long addressLength, i;

// get the type of the packet
switch (pRMAPPacket->GetPacketType())
{
    case RMAPPacket::RMAP_PACKET_TYPE_WRITE_COMMAND:
        cout << "The packet is an RMAP write command." << endl;
        break;

    case RMAPPacket::RMAP_PACKET_TYPE_WRITE_REPLY:
        cout << "The packet is an RMAP write reply." << endl;
        break;
}
```

```

    case RMAPPacket::RMAP_PACKET_TYPE_READ_COMMAND:
        cout << "The packet is an RMAP read command." << endl;
        break;

    case RMAPPacket::RMAP_PACKET_TYPE_READ_REPLY:
        cout << "The packet is an RMAP read reply." << endl;
        break;

    case RMAPPacket::RMAP_PACKET_TYPE_READ_MODIFY_WRITE_COMMAND:
        cout << "The packet is an RMAP read-modify-write command." << endl;
        break;

    case RMAPPacket::RMAP_PACKET_TYPE_READ_MODIFY_WRITE_REPLY:
        cout << "The packet is an RMAP read-modify-write reply." << endl;
        break;

    case RMAPPacket::RMAP_PACKET_TYPE_INVALID:
        cout << "The packet is of an invalid RMAP packet type." << endl;
        break;

    default:
        cout << "The packet type could not be obtained." << endl;
}

// get the target address of the packet
pAddress = pRMAPPacket->GetTargetAddress(&addressLength);
if ((!pAddress) || (!addressLength))
{
    cout << "No valid target address is present in the packet." << endl;
}
else
{
    // display the target address of the packet
    cout << "The packet has a target address of:";
    for (i = 0; i < addressLength; i++)
    {
        cout << " " << hex << (int)pAddress[i];
    }
    cout << endl;
}

// get the key of the packet
key = pRMAPPacket->GetKey();
cout << "The value of the key is " << hex << (int)key << endl;

```

RMAP Initiator Example Code

The examples above can be used to create an RMAP command packet to be transmitted, to check that a received packet is a valid RMAP packet, and to get the values of the fields in that packet. When combined with the information in the “**Transmitting and Receiving Packets Using the STAR-System C++ API**” application note, this is all the information required to write a simple RMAP Initiator.

The code below builds an RMAP read command packet and transmits it. It then waits for a reply packet to be received. If the reply packet is a valid RMAP packet, is a read reply, and is the reply for the command that was sent (checked using the transaction identifier), then the data read is displayed.

```

#include "device_list.hpp"
#include "device.hpp"
#include "channel.hpp"
#include "packet.hpp"
#include "transmit_operation.hpp"
#include "receive_operation.hpp"
#include "rmap_read_command_packet.hpp"
#include "rmap_packet_library.hpp"
using namespace stardundee::com::starsystem::general;
using namespace stardundee::com::starsystem::channels;
using namespace stardundee::com::starsystem::streamitems;
using namespace stardundee::com::starsystem::transferoperations;
using namespace stardundee::com::starsystem::rmap;

void RMAPInitiatorExample::DoRMAPInitiatorExample()
{
    DeviceList deviceList;
    U32 deviceCount;
    Device *pDeviceToUse;
    STAR_CHANNEL_MASK channelMask;
    Channel channel;
    Packet transmitPacket, *pReceivePacket;
    TransmitOperation transmitOp;
    ReceiveOperation receiveOp;
    TransferOperation::TransferStatus receiveStatus;
    RMAPReadCommandPacket readCommandPacket;
    RMAPPacket *pRMAPPacket;
    U8 targetAddress[] = {0, 254};
    U8 replyAddress[] = {254};
    const unsigned long targetAddressLength = sizeof(targetAddress);
    const unsigned long replyAddressLength = sizeof(replyAddress);
    const char alignment = 1, incrementAddress = 1;
    const U8 key = 0x20, extendedReadAddress = 0;
    const U16 transactionIdentifier = 0x1234;
    U16 transactionIdentifierReceived;
    const U32 readAddress = 0x106, readLength = 4;
    RMAPPacket::RMAPStatus status;
    U8 *pData;
    const char channelNumber = 1;
    U32 dataLength, i;

    // get the list of devices present
    deviceCount = deviceList.Populate();

    // if there are no devices present
    if (!deviceCount)
    {
        cout << "No devices present." << endl;
        return;
    }

    // display the number of devices present
    cout << deviceCount << " devices present." << endl;

    // get a pointer to the first device on the list
    // \note If the device list is freed or re-populated the pointer obtained will no
    //       longer be valid. Use DeviceList.CopyDeviceAt() instead to obtain a
    //       memory-allocated copy of the selected Device on the list (which must be
    //       freed when no longer required using the C++ delete operator)

```

```

pDeviceToUse = deviceList[0];

// get the channels on the device and check that the channel to be used exists
channelMask = pDeviceToUse->GetDeviceChannels();

// check for existence of channel
if (!(channelMask & (channelNumber << 1)))
{
    cout << "The device doesn't have a channel " << channelNumber << endl;
    return;
}

// open a bidirectional channel to channel 1 of the device
channel = pDeviceToUse->OpenChannel(Channel::DIRECTION_INOUT, channelNumber, 1);
if (!channel.IsOpen())
{
    cout << "Unable to open a channel to the device." << endl;
    return;
}

// build an RMAP read command packet to read 4 bytes
if (!readCommandPacket.CreateReadCommandPacket(targetAddress,
                                                targetAddressLength, replyAddress, replyAddressLength,
                                                incrementAddress, key, transactionIdentifier,
                                                readAddress, extendedReadAddress, readLength, alignment))
{
    cout << "Couldn't build the read command packet" << endl;

    // clean up resources created thus far, prior to exiting function
    channel.CloseChannel();

    // \note Some resources (such as transfer operations and packets) will
    //         automatically dispose of and/or destroy themselves if required when
    //         they go out of scope or are deleted, if they haven't already been
    //         'manually' disposed of by the user. Therefore, explicit calls to
    //         Dispose and Destroy methods need not be utilised during clean up
    //         stages and early exit stages if preferred, instead letting the
    //         objects clean up after themselves when the function exits and they go
    //         out of scope.
    //
    //         It is worth paying attention to the order in which objects may be
    //         disposed, however. For example, if a packet is destroyed while a
    //         transmit operation is still in the process of attempting to transmit
    //         the packet's data it may cause unexpected side effects.
    //
    //         This example makes explicit use of Dispose and Destroy methods for
    //         the purposes of illustration.

    // exit the function
    return;
}

// create a packet to transmit the RMAP read command, providing it with the RMAP
// packet's data and data length, terminated with an EOP.
if (!transmitPacket.CreatePacket((Address *)NULL,
                                (unsigned char *)readCommandPacket.GetPacketData(),
                                readCommandPacket.GetPacketLength(), Packet::EOP_TYPE_EOP))
{
    cout << "Couldn't create the stream item to be transmitted." << endl;
}

```



```
// clean up resources created thus far, prior to exiting function
readCommandPacket.Dispose();
channel.CloseChannel();

// exit the function
return;
}

// create a transmit operation to transmit the packet
if (!transmitOp.CreateTxOperation(&transmitPacket, 1))
{
    cout << "Couldn't create the transmit operation." << endl;

    // clean up resources created thus far, prior to exiting function
    transmitPacket.DestroyPacket();
    readCommandPacket.Dispose();
    channel.CloseChannel();

    // exit the function
    return;
}

// create a receive operation for receiving one packet, the RMAP reply
if (!receiveOp.CreateRxOperation(1, ReceiveOperation::RECEIVE_PACKETS))
{
    cout << "Couldn't create receive operation, exiting." << endl;

    // clean up resources created thus far, prior to exiting function
    transmitOp.DisposeTransferOperation();
    transmitPacket.DestroyPacket();
    readCommandPacket.Dispose();
    channel.CloseChannel();

    // exit the function
    return;
}

// start transmitting the packet
channel.SubmitTransferOperation(&transmitOp);

// start receiving a packet
channel.SubmitTransferOperation(&receiveOp);

// wait indefinitely on the reply packet being received
receiveStatus = receiveOp.WaitOnTransferOperationCompletion(
    TransferOperation::WAIT_INDEFINITELY);

// if failed to receive the packet
if (receiveStatus != TransferOperation::TRANSFER_STATUS_COMPLETE)
{
    cout << "Could not receive packet, error of " << receiveStatus << endl;

    // clean up resources created thus far, prior to exiting function
    receiveOp.DisposeTransferOperation();
    transmitOp.DisposeTransferOperation();
    transmitPacket.DestroyPacket();
    readCommandPacket.Dispose();
    channel.CloseChannel();
}
```

```

    // exit the function
    return;
}

// get the packet stream item received
pReceivePacket = (Packet *)receiveOp.GetTransferItem(0);

// no packet received
if (!pReceivePacket)
{
    cout << "Unable to get the stream item received." << endl;

    // clean up resources created thus far, prior to exiting function
    receiveOp.DisposeTransferOperation();
    transmitOp.DisposeTransferOperation();
    transmitPacket.DestroyPacket();
    readCommandPacket.Dispose();
    channel.CloseChannel();

    // exit the function
    return;
}

// extract RMAP packet from the received stream item
pRMAPPacket = RMAPPacketLibrary::GetRMAPPacketFromStreamItem(pReceivePacket,
                                                             1, &status);

// free the received packet stream item, now it has been processed
delete pReceivePacket;

// if failed to extract valid RMAP packet from received stream item
if (!pRMAPPacket)
{
    cout << "A valid RMAP packet was not received." << endl;

    // clean up resources created thus far, prior to exiting function
    receiveOp.DisposeTransferOperation();
    transmitOp.DisposeTransferOperation();
    transmitPacket.DestroyPacket();
    readCommandPacket.Dispose();
    channel.CloseChannel();

    // exit the function
    return;
}

// check the packet is a read reply
if (pRMAPPacket->GetPacketType() != RMAPPacket::RMAP_PACKET_TYPE_READ_REPLY)
{
    cout << "The packet received is not an RMAP read reply packet." << endl;
}
else
{
    // check the status in the reply packet is success
    status = pRMAPPacket->GetStatus();
    if (status != RMAPPacket::RMAP_STATUS_SUCCESS)
    {
        cout << "The reply packet contains an error status." << endl;
    }
}

```



```

else
{
    // check the transaction identifier is correct
    transactionIdentifierReceived = pRMAPPacket->GetTransactionID();
    if (transactionIdentifierReceived != transactionIdentifier)
    {
        cout << "The packet received has an unexpected transaction"
              << " identifier." << endl;
        cout << "Expected a value of " << hex << (int)transactionIdentifier
              << ", received a value of " << hex
              << (int)transactionIdentifierReceived << endl;
    }
    else
    {
        // get a pointer to the data in the reply
        pData = pRMAPPacket->GetData(&dataLength);

        // if no data
        if (!pData)
        {
            cout << "Unable to get the data in the RMAP read reply"
                  << " packet." << endl;
        }
        else
        {
            if (dataLength != readLength)
            {
                cout << "Unexpected data length in the RMAP read"
                      << " reply packet." << endl;
                cout << "Expected a length of " << (int)readLength
                      << ", received a length of " << (int)dataLength << endl;
            }

            // display the data in the reply
            cout << "The data read:";
            for (i = 0; i < dataLength; i++)
            {
                cout << " " << hex << (int)pData[i];
            }
            cout << endl;
        }
    }
}

// free the RMAP packet extracted from the received packet data
delete pRMAPPacket;

// dispose of the receive operation and transmit operation
// \note Transfer operation objects (such as ReceiveOperation
//       and TransmitOperation) will automatically dispose of any transfer
//       operation they hold if required when they go out of scope or are
//       deleted. Therefore, the following lines of code may be omitted if
//       preferred.
receiveOp.DisposeTransferOperation();
transmitOp.DisposeTransferOperation();

```

```

// destroy the transmit packet
// \note The Packet object will automatically destroy the packet it contains
//       if required when it goes out of scope or is deleted. Therefore,
//       the following line may be omitted if preferred.
transmitPacket.DestroyPacket();

// dispose of the RMAP read command packet
// \note The RMAPReadCommandPacket object will automatically dispose of the
//       RMAP packet it contains if required when it goes out of scope or
//       is deleted. Therefore, the following line may be omitted if
//       preferred.
readCommandPacket.Dispose();

// close the channel to the device
// \note The Channel object does NOT automatically close the channel it refers
//       to when it goes out of scope or is deleted. Therefore, it must be
//       closed by the user when no longer required as below:
channel.CloseChannel();
}

```

Note that it is possible to simplify this code using the `TransmitPacket()` and `ReceivePacket()` functions on the `Channel` object (see the “**Transmitting and Receiving Packets Using the STAR-System C++ API**” application note). This example does not make use of these functions as they are only useful in certain situations. Creating transmit and receive operations which are submitted to a channel for example, is more flexible and efficient.

